# Exploring AI-Powered Penetration Testing with PentAGI

EBAD JUNAID

BSCS 22046

APRIL 27 , 2026

## Question no : 1 – Agents Types

1. **Searcher**: This agent focuses on gathering information. During a test, it acts as the reconnaissance team, searching the internet and databases to learn as much as possible about the target system.

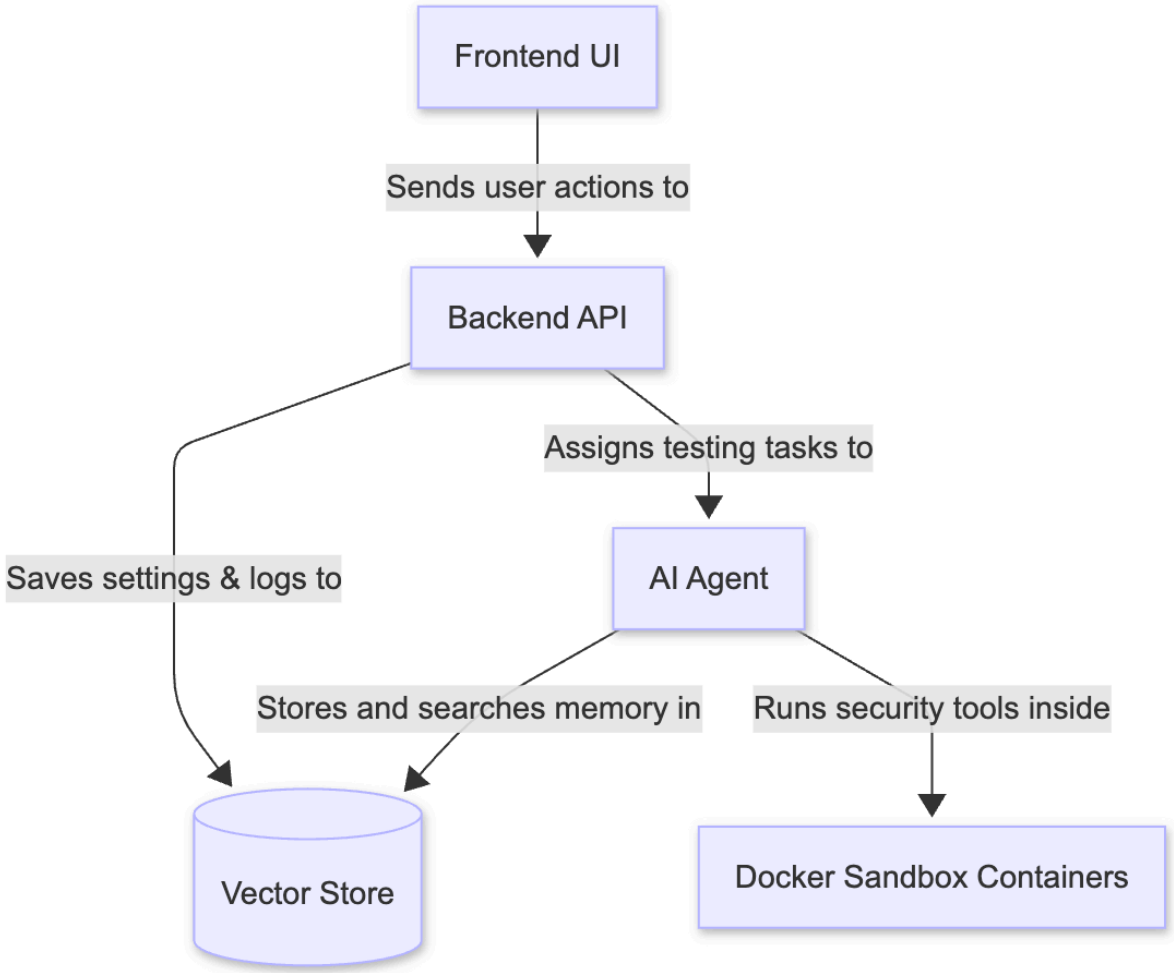
2. **Coder**: This agent handles writing and analyzing code. It writes custom scripts, develops specific exploit code, or reviews the target's software code to find hidden weaknesses.

3. **Pentester**: This is the active attacker agent. It is responsible for running security tools (like network scanners) and executing the actual attacks to find and exploit vulnerabilities.

4. **Adviser**: This agent acts as a mentor and strategist. It plans out the attack steps and gives expert advice to the other agents, especially when an attack fails and a new strategy is needed.

5. **Reflector**: This agent reviews mistakes and errors. If an agent gets stuck or a tool fails, the reflector analyzes what went wrong and helps guide the system back on track so it can finish the test successfully.

## Question 2 [Component Interaction Diagram]




## Question 3 [Vector Database vs Regular Database]

A vector database is more useful because it stores data based on its meaning rather than exact text. This allows the AI agent to use "semantic search" to find past experiences or attack patterns that are similar to its current situation, even if the exact words or commands are different. A regular relational database only looks for exact keyword matches, which is too strict and limited for an AI trying to learn from past penetration tests.

## Question 4 [Penetration Testing Plan]

| Testing Phase | Description | Agent Type | Built-in Tool |
|---|---|---|---|
| 1. Reconnaissance | Gather basic information and analyze the structure of the target website. | Searcher | scraper |
| 2. Scanning | Scan the server to find open network ports and basic vulnerabilities. | Pentester | nmap |
| 3. Exploitation | Attack the application to find and exploit weaknesses like database flaws. | Pentester | sqlmap |
| 4. Custom Exploitation | Write and run custom attack scripts if standard tools do not work. | Coder | terminal |
| 5. Reporting | Compile all the successful attacks and findings into a final document for the client. | Reporter | file |

## Question 5 [Vulnerability Report Elements]

A PentAGI vulnerability report should contain, at minimum, the following four elements:

**Executive Summary:** A simple, high-level overview of the test results that helps business managers and non-technical staff quickly understand the overall security, health and risks to their company.

**Technical Findings:** A detailed list of the specific vulnerabilities found in the web application, which tells the IT and security teams exactly what is broken and where the weaknesses are located in their system.

**Exploitation Guides:** Step-by-step proof showing how the PentAGI agent successfully attacked the system, proving to the client that the flaw is a real threat by showing exactly how a hacker could use it.

**Remediation Recommendations:** Clear instructions on how to fix or patch the discovered vulnerabilities, giving the developers the exact solutions they need to secure the application and prevent future attacks.

## Question 6 [Scope Breach and Responsibility]

If PentAGI's AI agent attacks a server outside the agreed scope, the penetration test must be halted immediately. The consultant must stop the tool, review the system logs to determine exactly what the AI accessed, and promptly notify the client about the scope breach.

Even though PentAGI is a highly autonomous system, the ultimate responsibility falls entirely on the human security consultant. In professional penetration testing, the authorization letter legally protects the tester only for the specifically listed targets. Attacking an unauthorized server can legally be considered a cyberattack, regardless of whether an AI or a human initiated it.

The consultant cannot blame the AI for "going rogue." It is the human operator's strict duty to establish technical boundaries before the test starts. This includes configuring network firewalls, applying strict routing rules, and properly isolating the Docker sandbox containers to physically prevent the tool from reaching off-limits servers. The consultant is fully accountable for controlling, monitoring, and securing their automated tools at all times to ensure they operate only within legal boundaries.

## Question 7 [Bearer Token Authentication]

**How Bearer Tokens Work:**

- **Issued:** A user logs into the PentAGI web interface and generates a new API token. The server creates a unique, secure text string and displays it to the user only once for safekeeping.
- **Transmitted:** Whenever the user wants to use the PentAGI API to start a task or check a report, they attach this token to their HTTP request. It is sent in the header formatted exactly as `Authorization: Bearer YOUR_TOKEN`.
- **Validated:** The PentAGI backend server receives the request and checks the token. It verifies the digital signature to ensure it is authentic and checks that the token has not passed its expiration date. If the token is valid, the server allows the action to proceed.

**Why it is more secure than a username and password:**

Using Bearer tokens is considered much more secure in an API context because it protects your primary login credentials. Instead of sending your actual password over the network for every single automated request which heavily increases the risk of it being intercepted or stolen you use a specialized token. These tokens are highly controllable and can be created with specific expiration dates, meaning they will automatically stop working after a set time. Furthermore, if a token is ever accidentally leaked, you can easily revoke or delete that specific token from the PentAGI settings immediately without needing to change your main account password or disrupt your other tools.

## Question 8 [API Token Security Practices]

Here are three security best practices from the PentAGI README, the specific attacks they prevent, and practical real-world scenarios of what could go wrong if they are ignored:

### 1. Never commit tokens to version control

- **The Practice:** Do not save API tokens directly inside your code files. Instead, you should always use secure environment variables or a dedicated secrets management system.
- **Defends Against:** Automated credential harvesting by malicious bots that constantly scan public code repositories for exposed secrets.
- **What could go wrong:** A developer writing a custom automation script hardcodes their PentAGI API token to save time and accidentally uploads it to a public GitHub repository. Within minutes, automated scanners find the token. Hackers immediately use this token to remotely control the PentAGI system, launching unauthorized attacks on the company's internal networks and exposing sensitive data.

### 2. Rotate tokens regularly

- **The Practice:** Set appropriate expiration dates for your tokens and establish a routine to replace them with newly generated ones on a regular basis.
- **Defends Against:** Long-term exploitation of compromised credentials and undetected, ongoing unauthorized access.
- **What could go wrong:** An IT team sets up a temporary testing server using a PentAGI API token that has no expiration date. Months later, that testing server is forgotten and eventually gets hacked through an outdated software plugin. The attackers find the active token and use it to silently download confidential vulnerability reports from PentAGI. If the token had been configured to expire after 30 days, the attackers' access would have been automatically blocked.

### 3. Use HTTPS only

- **The Practice:** Never send API tokens over standard, unencrypted internet connections. You must always ensure that your API traffic uses secure HTTPS protocols.
- **Defends Against:** Man-in-the-Middle (MitM) attacks, where an attacker secretly intercepts the data flowing between the user and the server.
- **What could go wrong:** A security consultant is traveling and decides to run a quick automated scan using their API token while connected to a hotel's public, unencrypted Wi-Fi network. An attacker staying in the next room is monitoring the local network traffic. Since the traffic is sent over plain HTTP, the attacker

easily reads the API token in plain text, copies it, and takes full control of the consultant's PentAGI testing environment.

**Question 9 [GraphQL Security Risk]**

One security risk that is more specific to GraphQL than to REST APIs is Deeply Nested Queries (Denial of Service).

**Explanation:** In a traditional REST API, the server controls exactly how much data is returned for each request. In GraphQL, the client gets to ask for exactly what data they want. An attacker can abuse this by sending a massive, deeply nested query (for example, asking for a user, then their tasks, then the users who created those tasks, then their tasks, repeating this many times). This forces the backend database to do a massive amount of work all at once, which can easily overload the server and crash the system and lead to a Denial of Service (DoS)

## Question 10 [Ethics of Autonomous Penetration Testing]

Full autonomy in penetration testing is not ethically appropriate because cybersecurity requires strict human judgment and control. While tools like PentAGI are incredibly powerful, letting an AI run completely on its own creates serious ethical and legal risks.

First, full autonomy threatens the principle of informed consent. Before a penetration test starts, a client gives written permission to test specific systems. If an AI is fully autonomous, it makes its own decisions and might use highly destructive attacks that the client never agreed to. A client cannot give true informed consent if no one, not even the human security consultant, knows exactly what the AI will decide to do next.

Second, an unsupervised AI easily leads to scope creep. AI agents are programmed to find paths to success. If an autonomous agent cannot break into the main target, it might independently decide to attack a different, connected network to find a backdoor. This scope creep means the AI is now illegally attacking computers that were not included in the original agreement. A software program lacks the moral understanding to stop itself when it crosses these boundaries.

Finally, there is the massive issue of accountability. If a fully autonomous AI makes a mistake and crashes a critical database or leaks sensitive customer records, who is to

blame? An AI program cannot be sued, fired, or put in jail. True accountability requires a human professional who takes responsibility for every single action.

In conclusion, while AI can greatly assist human testers, handing over total control is unethical. A human must always remain in charge to ensure the test respects the client's consent, stays within the agreed scope, and maintains legal accountability.

## Question 11 [Legal Violations]

Here are three specific laws or legal principles the student would violate by running PentAGI against the university network without permission:

**1. The Computer Fraud and Abuse Act (CFAA)** The Computer Fraud and Abuse Act is a major federal law in the United States that makes it illegal to intentionally access a computer system or network without proper authorization. By running an automated tool like PentAGI against the university's servers without getting explicit permission first, the student is actively breaking this law. In the eyes of the law, having a good motive or claiming that the attack was "just for learning" does not excuse the unauthorized access.

**2. The Computer Misuse Act (CMA)** The Computer Misuse Act is a strict law in the United Kingdom that criminalizes any unauthorized access to computer material, as well as any acts that might impair how a computer works. PentAGI is designed to run active security tools and exploit scripts that can easily crash servers, slow down networks, or alter data. Launching these powerful tools against the university network without the owner's consent is a clear criminal violation of this act, no matter what the student's intentions were.

**3. Cyber Trespass (General Legal Principle)** Cyber trespass is a general legal principle that treats digital networks similarly to physical property. Just as it is illegal to break into a locked school building at night just to "look around," it is illegal to enter a private digital network without consent. The university's network is private property. When the student uses PentAGI to scan or break into that network, they are committing digital trespassing. Having good intentions or claiming educational purposes is never a valid legal defense for unauthorized entry.

---